

TREE DELIVERY



Um jogo sério para o ensino de Árvores Binárias de Busca

Apresentação técnica • linguagem e portabilidade • proposta de extensão com Árvore AVL

Anna Clara Zordan Isabela Linhares Mariana Lima

Baseado em: Stultgens, P. — "Developing a game for undergraduate computer science students to learn data structures", MSc Software Engineering, Open University of the Netherlands.

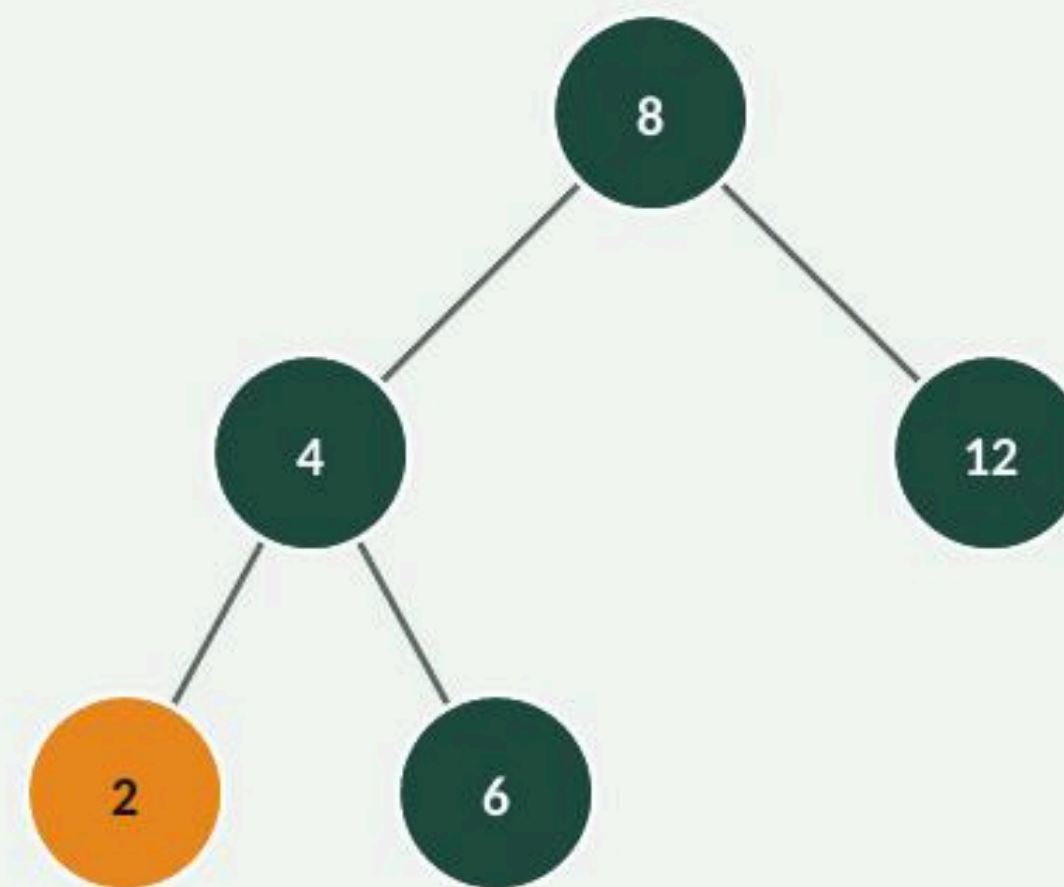
Tree Delivery: entregar pacotes é percorrer uma BST

Jogo 2D top-down de entregas: o jogador dirige um veículo pelas arestas e nós de uma Árvore Binária de Busca.

Objetivo: entregar cada pacote no nó correto, respeitando a propriedade da BST (esquerda < nó < direita).

Dificuldade progressiva, dicas e feedback calibrados por um teste de pré-conhecimento do jogador.

- Trabalha diretamente os erros clássicos de estudantes: inserir no lugar errado e remover nós com filhos.



*Pacote entregue no nó correto
(2 < 4, segue à esquerda)*

Modelo base e o lugar do jogo na literatura

Modelo base

- Estrutura de dados: Árvore Binária de Busca (BST) pura — sem qualquer garantia de balanceamento.
- Motor: Unity (jogo 2D/3D com física, câmera e UI prontas para um racing/delivery top-down).
- Origem: dissertação de mestrado com metodologia de teste de pré-conhecimento e avaliação pedagógica.

Limitações técnicas / didáticas

- Sem balanceamento: uma sequência de inserções ordenadas degenera a árvore em uma lista (altura $O(n)$).
- O jogador nunca vê consequência prática de uma árvore desequilibrada — custo de busca não é ensinado.
- Rotações, fator de balanceamento e reestruturação — tópicos centrais de ED II — ficam fora do escopo do jogo original.

Jogos correlatos citados na literatura de jogos sérios para estruturas de dados: DS-Hacker (Unity 3D, action-adventure para BST — FDG'20) mostra a mesma lacuna de balanceamento explorada aqui.

Da teoria da BST para regras de jogo

Conceito de BST	Regra de jogo
Esquerda < nó < direita	Rota de entrega correta: o pacote só é aceito no nó cujo valor respeita a comparação
Inserção em posição errada	Penalidade de tempo/pontuação — feedback visual imediato no nó errado
Remoção de nó folha	Entrega simples: remover o pacote sem reconexões
Remoção de nó com 1 filho	Minigame de reconexão de aresta (o filho “sobe” no lugar do nó removido)
Remoção de nó com 2 filhos	Desafio de sucessor: localizar o menor da subárvore direita antes de liberar a rota

Em que linguagem o jogo foi escrito?

C#

sobre o motor Unity

- Unity é o motor declarado pelo autor na página oficial do jogo.
- C# é a única linguagem de scripting de gameplay suportada pelo Unity.
- Lógica da BST implementada como classes/objetos (MonoBehaviour + estruturas próprias de nó).

Por que isso importa para a rubrica

- A linguagem é um detalhe de implementação. O que a rubrica cobra — reuso correto da literatura, rigor teórico da estrutura e inovação de gameplay — não depende de C# nem de Unity.
- Isso importa porque a próxima pergunta é exatamente essa: o jogo continuaria funcionando em outra linguagem?

Resposta curta: sim, funcionaria — a BST é um algoritmo, não um recurso do C# ou do Unity. Troca-se o motor gráfico (ex.: libGDX/JavaFX em Java), mas a lógica de inserção, busca e remoção é portada quase linha a linha.

De BST para Árvore AVL: a rota nunca fica torta

- Nova mecânica: “Zona de Rebalanceamento”. Sempre que uma entrega deixa o fator de balanceamento de um nó fora de $\{-1, 0, 1\}$, o trecho da pista se deforma (inclina/racha).
- O jogador deve executar a rotação correta — LL, RR, LR ou RL — dirigindo pela sequência certa de segmentos da pista antes do tempo acabar.
- Um HUD mostra o fator de balanceamento de cada nó em tempo real (barra verde = 0, amarela = ± 1 , vermelha = desequilíbrio).
- Pontuação bônus por manter a árvore balanceada ao longo do nível; penalidade progressiva por deixá-la degenerar.

